WHU
Otto Beisheim School of Management

From Raw Data to Finished Paper: Building a Reproducible Research Pipeline

Doctoral Orientation Week

Dr. Victor van Pelt

WHU – Otto Beisheim School of Management

August 5, 2026

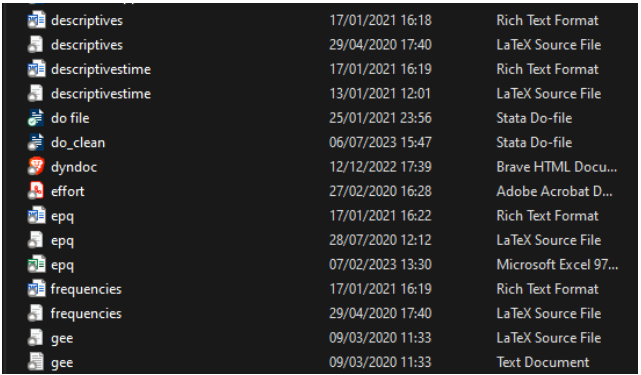
Opening Question

Let's say you wrote a research paper and your laptop died. How long would it take you to reproduce your analyses with a new laptop?

1. Less than one hour (easy!)
2. Between one hour and a day (It's possible but painful)
3. More than one day (I'd be stuck)

Why should YOU care about Data Management?

Storing data, code, and analyses can quickly turn into a mess:



descriptives	17/01/2021 16:18	Rich Text Format
descriptives	29/04/2020 17:40	LaTeX Source File
descriptivestime	17/01/2021 16:19	Rich Text Format
descriptivestime	13/01/2021 12:01	LaTeX Source File
do file	25/01/2021 23:56	Stata Do-file
do_clean	06/07/2023 15:47	Stata Do-file
dyndoc	12/12/2022 17:39	Brave HTML Docu...
effort	27/02/2020 16:28	Adobe Acrobat D...
epq	17/01/2021 16:22	Rich Text Format
epq	28/07/2020 12:12	LaTeX Source File
epq	07/02/2023 13:30	Microsoft Excel 97...
frequencies	17/01/2021 16:19	Rich Text Format
frequencies	29/04/2020 17:40	LaTeX Source File
gee	09/03/2020 11:33	LaTeX Source File
gee	09/03/2020 11:33	Text Document

Why should YOU care about Data Management?

Story Time!

But there is more at stake!

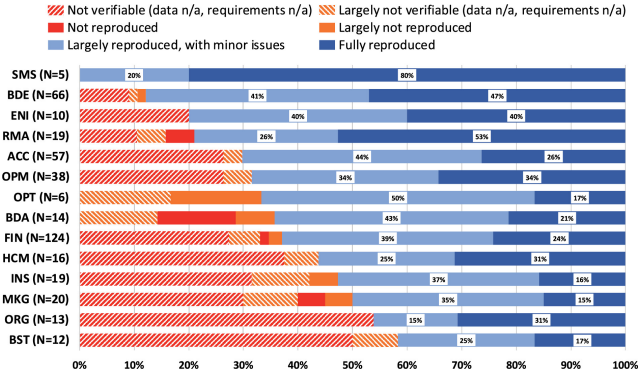
In 2019, *Management Science* introduced a Data and Code Disclosure Policy:

"Authors of accepted papers… must provide… the data, programs, and other details of the experiment and computations sufficient to permit replication."

A 2024 study examined nearly 500 articles published before and after this policy was introduced to see if the results could, in fact, be reproduced (Fišar et al. 2024).

What share of these articles do you think were largely reproducible?

But there is more at stake!



Across all articles under the new policy, 68% could be reproduced. When data and software were accessible, over 95% could (Fišar et al. 2024)

But there is more at stake!

- Even under the policy, nearly one in three papers fails the reproducibility test.
- Before the policy, it was worse: only 12% of articles shared any materials at all.
- This is why good data and code management is not only a “nice to have.”
- It’s also essential for credible research and for us to trust the insights it produces.
- But this does not just depend on how professors manage their data and code.
- It all starts with:

The main goals of this session:

- Share a few basic principles, techniques, and tools. Inspired by:
  - **Code and Data for the Social Sciences** (Gentzkow and Shapiro)
  - **How to keep your research projects organized** (de Kok)
  - **Good Enough Practices in Scientific Computing** (Wilson et al.)
  - My own mistakes and personal experience
- Let's make this interactive: Interrupt, ask questions, and tell me about your experience.
- Disclaimer: Largely through the lens of a quantitative researcher.
- All materials for this talk are available: **Example Files** and **Slides**

What will we cover in this session?

1. Directory structure: How to organize your research project
2. Version control: Using Git and repositories (example using GitHub)
3. Integrating multiple languages: Example using Quarto
4. AI agents and systems: Using them without losing reproducibility, plus advice for your own setup

1. Directory Structure

1. Directory Structure: Fundamental Principles

Following principles is critical, especially for projects with lots of data and code:

- Anyone can run the code anywhere and anytime, regardless of location.
- Anyone can understand the code and data without much effort.
- Anyone can change one number or one step and rerun, with no hardcoded values to hunt down.

1. Directory Structure: Structure the flow

- Use a flow-inspired folder structure:
  - E.g., formatting files -> generating variables -> conducting analyses -> generating output.
- Leave the raw data untouched: You load it and use it to produce output:
  - E.g., raw data files -> input files -> process files -> output files.
- Use relative paths (e.g., "*../0_data/data.csv*") and not direct paths (e.g., "*C:/user[name]/Documents/research/project_1/0_data/data.csv*").

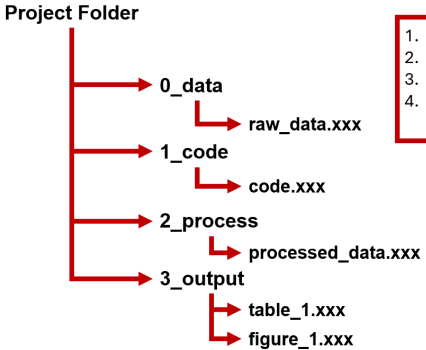
> Anyone should be able to delete everything except the raw data and the code, and rebuild your whole project.

1. Directory Structure: Code so others can run it

- Make your code deterministic: set a seed for random and stochastic processes.
- Pin your environment: "*version*" in Stata, *renv* in R.
- Use plenty of comments to explain what the code is doing.
- Be as descriptive as you can in folder and file names; avoid special characters.
- Prefer tool-agnostic file formats: a "*.csv*" opens anywhere, for anyone.
- Use environment variables for credentials and software paths.
- Follow a template for a research pipeline (**TIER Protocol**); the example repository follows its concept.
  - E.g., a codebook ("*0_data/codebook.md*") defines each variable and the data's origin.

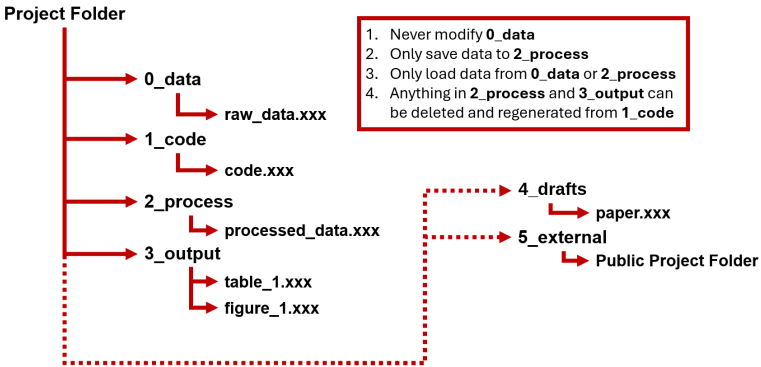
> Write for the stranger who inherits the project, because in two years that stranger is you.

1. Directory Structure: A simple starting point



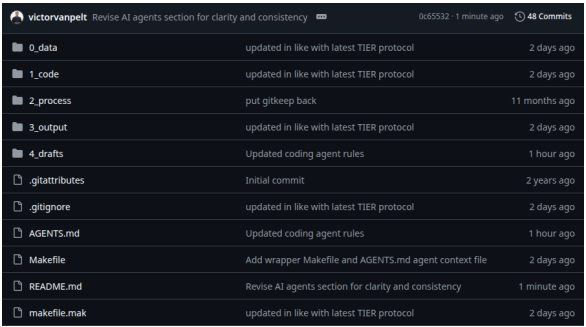
1. Never modify **0_data**
2. Only save data to **2_process**
3. Only load data from **0_data** or **2_process**
4. Anything in **2_process** and **3_output** can be deleted and regenerated from **1_code**

1. Directory Structure: Additional folders



1. Directory Structure: A working example

Let's take a closer look at how this works in practice (using Stata, R, and Quarto):



Clone this repository!

github.com/victorvanpelt/research_pipeline_example

1. Directory Structure: Stata do-file under “1_code/”

1_code/code.do (excerpt)

```
//clear everything to ensure nothing came before
clear all

//go to the project root
capture cd 1_code   // if we're at the project root, step into 1_code (otherwise nothing)
cd ..               // now we're guaranteed to be at the project root
// Forward slashes work on Windows, macOS, and Linux.

//import raw data and store an untouched copy in the process-folder (also handy for merging)
import delimited "0_data/gen_ai_earnings.csv", clear
save "2_process/gen_ai_earnings.dta", replace

//import the raw copy, transform the data, and save the edited data
use "2_process/gen_ai_earnings.dta", clear

gen earnings_scaled = earnings / 10
drop if earnings_scaled < 0

save "2_process/edit_gen_ai_earnings.dta", replace

//import the edited data and run the analysis to generate output
use "2_process/edit_gen_ai_earnings.dta", clear

//Run a basic regression with robust SE and store results (run "ssc install estout" once first!)
eststo: regress earnings_scaled gen_ai, r

//Write a LaTeX table to 3_output that the paper and slides can \input.
//"booktabs" produces \toprule/\midrule/\bottomrule, matching the R output.
//"nonumbers" drops esttab's default "(1)" header row and "coeflabels" renames
//"_cons" to "Constant", so this table matches the one code.r writes.
esttab using "3_output/table_1.tex", replace booktabs ///
b(3) se(3) star(* 0.10 ** 0.05 *** 0.01) stats(r2 F p N, fmt(3 3 3 0)) ///
nonumbers coeflabels(_cons Constant) ///
title(TABLE 1 - REGRESSIONS OF SCALED EARNINGS ON GENERATIVE AI) ///
nonotes addnotes("Robust standard errors in parentheses. * p<0.10, ** p<0.05, *** p<0.01.")
eststo clear
```

The same analysis also ships in R (“*code.r*”) and Quarto (“*code.qmd*”)

1. Directory Structure: Reproducing all results instantly

How would you reproduce all results of your current project?

1. One command rebuilds everything (a makefile or master script)
2. A script per step, run by hand in the right order
3. Point and click, from memory

1. Directory Structure: makefile.mak in root

A makefile states which outputs depend on which code and data, and how to rebuild them

makefile.mak

```
# ============================================================================
#  Research pipeline - build tasks
#
#  Usage:
#    make           run the R analysis, then build the appendix, paper, slides
#    make r         run the R analysis          (1_code/code.r)
#    make stata     run the Stata analysis      (1_code/code.do)
#    make quarto    render the self-contained Quarto report (1_code/code.qmd)
#    make appendix  render the data appendix    (4_drafts/data_appendix_example.qmd)
#    make paper     render the paper            (4_drafts/paper_example.qmd)
#    make slides    render the slides           (4_drafts/presentation_example.qmd)
#    make clean     delete everything the pipeline generates
#    make help      list the targets
#
#  Tools are overridable, e.g.:   make r RSCRIPT=/path/to/Rscript
#                                 make stata STATA=StataSE-64 STATAFLAG=/e
#  Forward slashes work on every OS. On Windows, run this from Git Bash / MSYS2, which provide make together with rm, mv, and find.
# ============================================================================

# Tools (override on the command line or via environment variables)
RSCRIPT   ?= Rscript
STATA     ?= stata-se
STATAFLAG ?= -b
QUARTO    ?= quarto

.DEFAULT_GOAL := all
.PHONY: all r stata quarto appendix paper slides clean help

# Default build: run the R analysis, then render the appendix, paper, slides.
all: r appendix paper slides

# --- Analysis engines (pick whichever one you use) ---------------------
r:
	$(RSCRIPT) --vanilla 1_code/code.r

# On Windows Stata the batch flag is "/e":  make stata STATAFLAG=/e
stata:
	$(STATA) $(STATAFLAG) do 1_code/code.do
	@rm -f code.log 1_code/code.log
```

1. Directory Structure: makefile.mak in root

You run “*make*” in the CLI to rebuild everything, or a single step with “*make r*”, “*make stata*”, “*make quarto*”, “*make appendix*”, “*make paper*”, or “*make slides*”

Terminal — research_pipeline_example

```
$ make
Rscript --vanilla 1_code/code.r
quarto render 4_drafts/data_appendix_example.qmd
quarto render 4_drafts/paper_example.qmd
quarto render 4_drafts/presentation_example.qmd
```

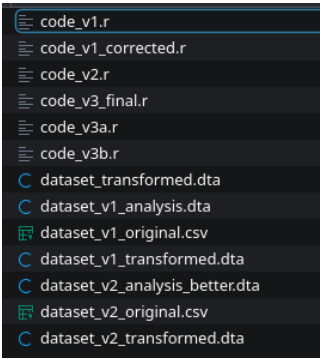
1. Directory Structure: Is your code slow?

- Does your code:
  - Take a long time to run?
  - Require too much memory?
- Ask your supervisor for a new laptop.
- Otherwise, consider looking into running your code remotely:
  - A “supercomputer,” “grid,” or “server” allows you to run your code using much more powerful hardware.
  - Ask about your school’s research computing services.

2. Version Control

2. Version control: Opening question

Whose folders look something like this?



2. Version control

Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later.

2. Version control: How to use version control

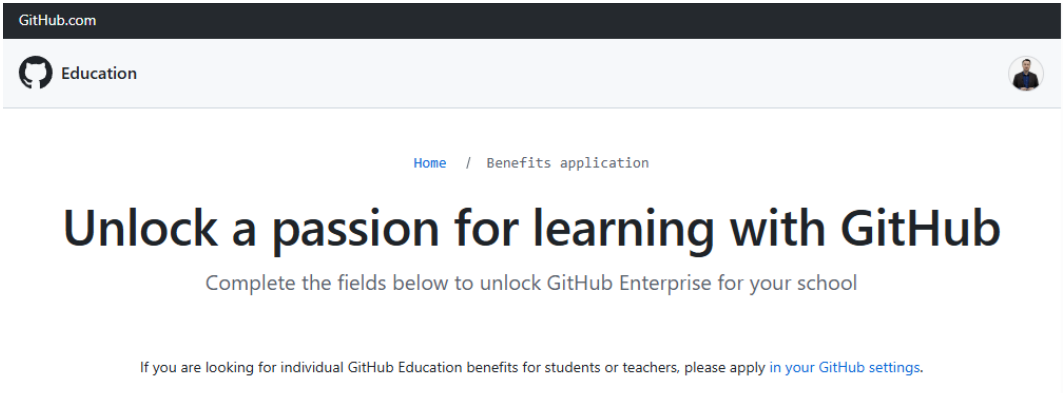
- Many cloud services have some forms of version control (e.g., Dropbox and OneDrive), but they can be too simplistic and unreliable.
- The most widely used software is called **git**
- You can run **git** locally using the command line (CLI), but it is easier to use an online provider.
- Saving your version control online reduces the likelihood you lose stuff.
- Three major **git** providers:
  - GitHub –> The best choice!
  - Bitbucket
  - GitLab
  - (EU nonprofit alternative: Codeberg)

2. Version control: What is GitHub?

- GitHub hosts your git repositories online and adds collaboration tools around them.
- Why use it?
  - Clean and easy-to-use interface
  - Works with any language and file type
  - GitHub Desktop application is very simple and convenient
- Bonus feature: you can use GitHub Pages to host your website for free! See mine **www.victorvanpelt.com**

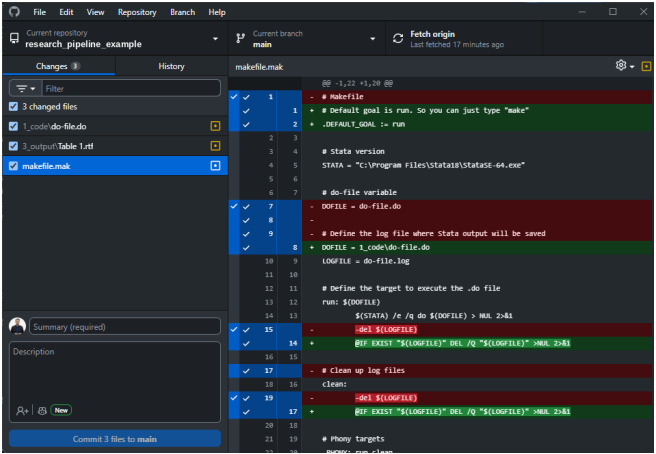
2. Version control: GitHub Education

As a verified student, you get GitHub Pro and the Student Developer Pack for free **here**.



2. Version control: GitHub Desktop

Download GitHub Desktop for Windows or macOS **here**.



2. Version control: Basic Workflow

- First time:
  - Create a new repository on GitHub
  - Clone it to your computer (Copy URL and enter in GitHub Desktop)
- When you start working:
  - Sync with GitHub first: “Pull” changes
- When making changes:
  - Sync with GitHub first: “Pull” changes
  - Create commit (add summary and descriptions)
  - Push commit to GitHub.
- Use a README.md file to state the project’s title and a brief description of the repository.

2. Version control: .gitignore files

- There are lots of things you might not want to sync with GitHub
  1. Private or non-public data
  2. Personal credentials
  3. “Byproduct” files
- Think carefully about what you sync with GitHub.
- A .gitignore file allows you to select files that should automatically be ignored by git.

2. Version control: .gitignore syntax

- The basic syntax can be found **here.**
- Some common expressions:
  - # -> comment
  - * -> any file name
  - ** -> any folder depth
  - / at the end -> folder
  - ! -> keep (don't ignore)

2. Version control: .gitignore example

The example repository ignores everything by default, then allows back what should be shared: the safest default for private data

.gitignore

```
# Ignore everything by default...
*

# ...but keep the project scaffolding
!.gitignore
!.gitattributes
!makefile.mak
!Makefile
!AGENTS.md
!README.md

# 0_data: keep the folder, the shared synthetic example CSV, and its codebook.
# Real raw data is intentionally NOT shared here (add it locally; it stays ignored).
!0_data/
!0_data/.gitkeep
!0_data/gen_ai_earnings.csv
!0_data/codebook.md

# 1_code: keep the folder and all code (the scripts are the point of the repo),
# but not the artifacts rendering code.qmd leaves behind
!1_code/
!1_code/**
1_code/*.pdf
1_code/*.html
1_code/*_files/

# 2_process: keep the folder but none of its intermediate contents
!2_process/
!2_process/.gitkeep

# 3_output: keep the folder and all shared outputs (tables, figures)
!3_output/
!3_output/**

# 4_drafts: keep the sources, the materials, and the rendered PDFs;
# render intermediates (.tex, .html, .aux, *_files/, ...) stay ignored.
!4_drafts/
!4_drafts/*.qmd
!4_drafts/*.bib
!4_drafts/*.pdf
!4_drafts/materials/
!4_drafts/materials/**
```

2. Version control: Why should you care?

- Go back to any version at any time!
- Journals and institutions are increasingly requesting access to your research materials (i.e., code, instrument, and data).
- At the very least, they want to ensure it exists.
- At the very most, they have your code checked line by line. Two journals in my area:
  - *Management Science* has verified reproducibility since 2019, partly through a third-party agency (CASCaD). Since 2025, a $79 submission fee helps fund these checks.
  - *Journal of Accounting Research* expects a full description of data provenance and processing.
- With the structure from chapter 1, compliance is nearly free.

3. Integrating Multiple Languages

3. Integrating multiple languages: What's the remaining challenge?

- So, now you will have:
  1. A good directory structure
  2. Version control.
- A remaining challenge is that we use different software, coding languages, and files.
  - Python, R, Markdown, HTML, Office, Stata, LaTeX, etc.
  - Tables, datasets, code, and docs.
- There have been efforts over the past decade to put all processes under one umbrella.
- One system can integrate everything into one process: Quarto.

3. Integrating multiple languages: What is Quarto?

- Quarto is an open-source scientific and technical publishing system
- Quarto is not only used for data. It can use data to produce a wide variety of outputs:
  - Articles and papers
  - Presentations (this presentation is made using Quarto)
  - Dashboards
  - Websites
- Runs R (Knitr), Python and Julia (Jupyter), and Observable JS next to Markdown and LaTeX.
- Quarto 2 (late 2026) stays backward compatible; your .qmd files keep working.

3. Integrating multiple languages: How to get started?

Setup:

1. Install Quarto from the website: **https://quarto.org/docs/get-started/**.
2. Choose a coding environment (I recommend **VS Code**).
3. Install the **Quarto VS Code Extension** in VS Code.

Usage:

- Quarto uses .qmd files. Check the **guide**.
- You can produce output in formats, such as .pptx, .docx, .pdf and integrate R and Stata code.
- The example repository uses Quarto to render a paper, a presentation, and a data appendix (“*4_drafts/*”) that pull their numbers and tables straight from the pipeline.
- It also contains a self-contained literate report (“*1_code/code.qmd*”).

3. Integrating multiple languages: Quarto Presentation Example

1_code/code.qmd

```yaml
---
title: "Quarto Example: A Report Built at Render Time"
author: "Victor van Pelt"
format:
  beamer:
    theme: Madrid
    fontsize: 9pt
---
```

Specify in the ”YAML” the type of qmd file

1_code/code.qmd

```r
```{r}
#| include: false

# Use CRAN mirror (needed when running via Rscript / make / quarto)
options(repos = c(CRAN = "https://cloud.r-project.org"))

# Move to the project root whether we render from the root or from 1_code.
# knitr resets the working directory after every chunk, so we store ROOT and
# build absolute paths with file.path(ROOT, ...) instead of relying on setwd().
if (dir.exists(file.path(getwd(), "1_code"))) setwd(file.path(getwd(), "1_code"))
setwd("..")
ROOT <- getwd()

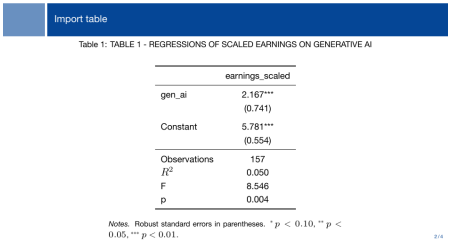
need <- c("sandwich", "tibble", "modelsummary")
to_install <- setdiff(need, rownames(installed.packages()))
if (length(to_install)) install.packages(to_install)  # repos taken from options()
invisible(lapply(need, require, character.only = TRUE))

# One self-contained pipeline: import -> transform -> analyse (no other file needed)
df <- read.csv(file.path(ROOT, "0_data", "gen_ai_earnings.csv"))
df$earnings_scaled <- df$earnings / 10
df <- df[df$earnings_scaled >= 0, , drop = FALSE]

mod <- lm(earnings_scaled ~ gen_ai, data = df)
vc  <- sandwich::vcovHC(mod, type = "HC1")          # robust SE, matches Stata's ", r"
```
```

Use R as you would in code.r

3. Integrating multiple languages: Quarto Presentation Example



One pipeline, three outputs (report, paper, slides), and the numbers always in sync.

3. Integrating multiple languages: makefile.mak extension

"*make appendix*", "*make paper*", and "*make slides*" render the Quarto drafts from the pipeline's output; "*make quarto*" renders the self-contained report

makefile.mak (Quarto targets)

```
quarto:
    $(QUARTO) render 1_code/code.qmd

# --- Manuscript, slides, and data appendix (consume 2_process and 3_output)
# Run `make r` (or `make stata`) first so the inputs they read exist.
appendix:
    $(QUARTO) render 4_drafts/data_appendix_example.qmd

paper:
    $(QUARTO) render 4_drafts/paper_example.qmd

slides:
    $(QUARTO) render 4_drafts/presentation_example.qmd

# --- Housekeeping -------------------------------------------------------
# Delete everything in 2_process and 3_output except the .gitkeep files,
# plus stray render artifacts, so a fresh `make` reproduces every output.
clean:
    @find 2_process 3_output -type f ! -name '.gitkeep' -delete
    @rm -f 1_code/code.pdf 4_drafts/*.pdf *.log 1_code/*.log
    @echo "Cleaned 2_process, 3_output, and render artifacts."

help:
    @echo "Targets: all (default), r, stata, quarto, appendix, paper, slides, clean"
```

4. AI Agents and Systems

4. AI agents: Opening question

You share a research repository with two coauthors. Each of you has an AI agent working in it, and each agent edits files and runs commands on its own. What could go wrong?

4. AI agents: From chatbots to agents inside your project

- Chat window: you paste, it answers, you paste back
- Agentic system: reads your files, edits code, runs commands
- It iterates until the pipeline runs
- Examples: Claude Code, OpenAI Codex, Hermes, Cursor

```
Terminal — an agent working in research_pipeline_example
$ claude "Add a robustness check: winsorize scaled
  earnings at the 1st/99th percentiles before
  the main regression."

* Reading 1_code/code.r
* Editing 1_code/code.r (+5 lines)
* Running: make clean && make
    Rscript --vanilla 1_code/code.r
    quarto render 4_drafts/data_appendix_example.qmd
    quarto render 4_drafts/paper_example.qmd
    quarto render 4_drafts/presentation_example.qmd

Done. Review my change with: git diff 1_code/code.r
```

4. AI agents: The problem, a shared repo full of agents

- Every coauthor brings a different agent, or none at all
- Agents act autonomously: they edit files and run commands
- Tools change monthly; tool-specific setups rot fast
- Without shared rules: edited raw data, patched outputs, results without code

> Your agent setup is personal. The rules it follows are shared.

4. AI agents: The one rule

- An agent can hand you a merged dataset, table, or figure
- No code behind it: nothing to rerun, check, or fix
- Your pipeline reruns: every step is code, written down, versioned
- An AI act cannot: same prompt, different answer, chat gone tomorrow
- So: everything AI contributes arrives as code in the repository

> AI may write the code that does the work. It must never **do** the work.

4. AI agents: The solution, a shared rulebook called AGENTS.md

- One file in the repo root states the rules for any agent
- Scope: help in "*1_code*" and "*4_drafts*" only
- "*0_data*" stays read-only; "*2_process*" and "*3_output*" are rebuilt, never hand-edited
- It may run *make*; it may never commit or push
- It must verify before it reports: "*it should work*" is not verification
- The rules from chapters 1-3, written down for the agent

AGENTS.md (rules; build targets and verification checklist follow)

```
# AGENTS.md — context for AI coding agents

This repository is a template for a reproducible research pipeline. Data flows through numbered
folders; everything downstream of the raw data is regenerated by code.

## Rules

- Never modify the raw data in `0_data/`. It is read-only. The exception is
  `0_data/codebook.md`, which is hand-maintained documentation: update it whenever the data
  change.
- Never edit files in `2_process/` or `3_output/` by hand (including
  `3_output/data_appendix/`): they are build artifacts, regenerated by the pipeline.
- Help the user make changes in `1_code/` (analysis), `4_drafts/` (paper, slides, and data
  appendix sources), and `0_data/codebook.md` only.
- Before a change that touches more than one file, say which files you will edit and what will
  change in each. Then edit.
- Never run `git commit` or `git push`. Leave changes in the working tree for the maintainer to
  review and commit.
- Do not commit credentials; use environment variables.
- Remove any scratch or helper files you create in this repository after you're finished. The
  folder structure is fixed; do not add folders or top-level files.
```

> Your pipeline does not need to trust the agent, only to verify what it changed.

4. AI agents: The agent edits code, you review the diff

- Prompt: winsorize scaled earnings at 1st/99th percentile
- The agent edits "*1_code/code.r*" directly
- You run git diff before committing
- Check: only "*1_code*" touched, and the step sits before the regression
- Check: "*0_data*" and "*3_output*" untouched?
- Try it yourself: the repo's README ships this exercise

```
git diff — 1_code/code.r

diff --git a/1_code/code.r b/1_code/code.r
index 57f4b0c..5414ac9 100644
--- a/1_code/code.r
+++ b/1_code/code.r
@@ -35,6 +35,10 @@ df <- readRDS(path_proc)
 df$earnings_scaled <- df$earnings / 10
 df <- df[df$earnings_scaled >= 0, , drop = FALSE]   # drop the negative scaled earnings

+# Robustness: winsorize scaled earnings at the 1st and 99th percentiles
+q <- quantile(df$earnings_scaled, probs = c(0.01, 0.99))
+df$earnings_scaled <- pmin(pmax(df$earnings_scaled, q[1]), q[2])
+
 saveRDS(df, path_edit)

 # 3. Reload the edited data and run the analysis
```

AGENTS.md: "Help the user make changes in 1_code/ (analysis), 4_drafts/ (paper, slides, and data appendix sources), and 0_data/codebook.md only."

4. AI agents: The agent runs make, you rerun it yourself

- The agent may run make and fix its own errors
- Your rule: "*make clean*", then "*make*"
- Plain "*make*" rebuilds R only; other engines need "*make stata*", "*make quarto*"
- On your machine, from raw data to drafts
- Numbers flow from the pipeline, not a chat

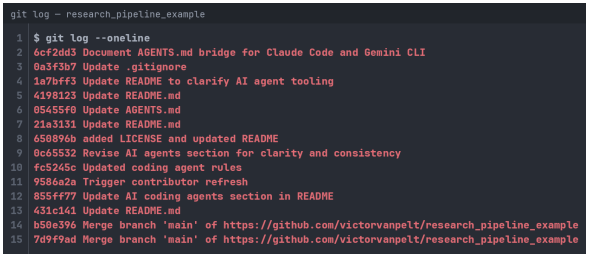
Terminal — rerunning the pipeline yourself

```
$ make clean
Cleaned 2_process, 3_output, and render artifacts.
$ make
Rscript --vanilla 1_code/code.r
quarto render 4_drafts/data_appendix_example.qmd
Output created: data_appendix_example.pdf
quarto render 4_drafts/paper_example.qmd
Output created: paper_example.pdf
quarto render 4_drafts/presentation_example.qmd
Output created: presentation_example.pdf
```

AGENTS.md: "A change is done only when make clean followed by make completes without errors and every number in the 4_drafts/ outputs comes from the rerun pipeline, not from hand edits."

4. AI agents: Commit the change, keep the trail

- One change, one commit; the message says what changed and why
- The commit history becomes your log of what the agent did
- Bigger changes: agent works on a branch, you review the pull request
- This is how you check an agent's work weeks later



AGENTS.md: "Never run git commit or git push. Leave changes in the working tree for the maintainer to review and commit."

4. AI agents: The researcher stays in the loop

- AI assists; it is never author, analyst, or judge
- Verify everything it touched: sources, numbers, code, interpretation
- The pipeline hands you the gates: read the diff, rerun *make*, check the trail
- A gate you wave things through is decoration, not verification

> Stay in the loop: nothing an AI produced enters your paper unverified.

4. AI agents: When AI touches data, it becomes generated data

- Raw data stays untouched in "*0_data*": the record of what actually happened
- Once AI cleans, codes, merges, or edits a data file, the output is generated data: the model's choices are inside it
- Structuring data is a research act, even when software performs it
- Keep every AI transformation as code in "*1_code*": rerunnable, reviewable, visible in the diff
- Disclose the touch: tool, task, your verification, and whether AI output entered the final data

> Data an AI touched is generated data, not naturally occurring data. Keep the transformation in code, and disclose it.

4. AI agents: Rules of engagement

Do

- Ask for code: merges, robustness checks, engine translations, makefile targets
- Let the agent work inside the repo, under git
- Review every diff before committing
- Rerun the pipeline before trusting a number
- Set the rules once, in "*AGENTS.md*"

Never

- A dataset, table, or coefficient straight from an AI
- A citation you did not verify yourself
- Confidential or licensed data into a chat
- Edits to "*0_data*" or "*3_output*", by you or the agent
- Undisclosed AI use in a submission

AGENTS.md: "Never modify the raw data in 0_data/. It is read-only. The exception is 0_data/codebook.md, which is hand-maintained documentation: update it whenever the data change."

4. AI agents: Your own setup, skills and agents

- The repo ships shared rules; your own setup is personal
- A skill: a written procedure that runs where you watch
- An agent: works out of sight, returns only a result
- Claude Code, OpenAI Codex, and Hermes name them differently

> A skill shows you the process. An agent hands you a result.

4. AI agents: Why research leans skill-first

- Skill-first: skills carry the process, agents get disposable subtasks
- Agent-first: thin skills, autonomous agents do the main work
- A finding needs a defensible account of how it was obtained
- A skill commits you to the steps before the work runs
- A quietly skipped check looks exactly like one that passed

> Automation is not delegation. Automate the boring steps; never delegate the judgment.

4. AI agents: Preliminary advice for your own setup

- Pick one CLI tool; expect to replace it
- Write AGENTS.md rules before any automation
- Build your first skill around a workflow you repeat
- Hand agents only work whose reasoning you can discard
- Keep early idea work loose; procedure suppresses variance there

> Keep the researcher in the loop. Every judgment call still lands on your desk.

Summary and Wrap-up

- Across all fields of science, reproducibility has been under threat (Open Science Collaboration 2015; Baker 2016; Camerer et al. 2016; Hail, Lang, and Leuz 2020; Fišar et al. 2024)
- Good data management is vital to ensure results are reproducible.
- You, as a WHU doctoral student, are vital:
  1. Maintain a good directory structure
  2. Use version control
  3. Integrate multiple languages (try Quarto)
  4. Let AI write pipeline code; never let it do the research for you
     - Prefer a skill-first setup: keep the researcher in the loop
     - If AI touched the data, it is generated data: keep the transformation in code and disclose it
- If you don't want to do it for science, then do it to save yourself lots of time and headache.

Download Example Files (github.com/victorvanpelt/research_pipeline_example)

Download Slides (github.com/victorvanpelt/research_pipeline_slidedeck)

Good data management is just the start...

- Good data management is not enough.
- Ideally, we also need:
  - Research material sharing (data, instruments, and code)
  - Pre-registration
- Many journals and institutions offer ways to pre-register and collect all research materials (not just data and code) in one place.
  - You can sync your repositories as well!
- My recommendation is to use the **Open Science Framework (OSF)**
- Steal a template: the **TIER Protocol** and the **AEA template README**

Thank you!

Dr. Victor van Pelt
Professor of Accounting
Finance and Accounting Group
WHU – Otto Beisheim School of Management

Campus Vallendar, Burgplatz 2, 56179 Vallendar, Germany
Tel.: +49 (0)261 6509 483
Victor.vanPelt@whu.edu
https://www.victorvanpelt.com

References I

Baker, Monya. 2016. “1,500 Scientists Lift the Lid on Reproducibility.” *Nature* 533 (7604): 452–54. https://doi.org/10.1038/533452a.

Bloomfield, Robert, Mark W. Nelson, and Eugene Soltes. 2016. “Gathering Data for Archival, Field, Survey, and Experimental Accounting Research.” *Journal of Accounting Research* 54 (2): 341–95. https://doi.org/10.1111/1475-679X.12104.

Camerer, Colin F, Anna Dreber, Eskil Forsell, Teck-Hua Ho, Jürgen Huber, Magnus Johannesson, Michael Kirchler, et al. 2016. “Evaluating Replicability of Laboratory Experiments in Economics.” *Science* 351 (6280): 1433–36. https://doi.org/10.1126/science.aaf0918.

Fišar, Miloš, Ben Greiner, Christoph Huber, Elena Katok, and Ali I. Ozkes. 2024. “Reproducibility in Management Science.” *Management Science* 70 (3): 1343–56. https://doi.org/10.1287/mnsc.2023.03556.

Hail, Luzi, Mark Lang, and Christian Leuz. 2020. “Reproducibility in Accounting Research: Views of the Research Community.” *Journal of Accounting Research* 58 (2): 519–43. https://doi.org/10.1111/1475-679X.12305.

References II

Open Science Collaboration. 2015. “Estimating the Reproducibility of Psychological Science.” *Science* 349 (6251): aac4716. https://doi.org/10.1126/science.aac4716.